Scenario 1 Master Blueprint: Toast Tables HLD

1. System Requirements & Context

Goal: Design a highly concurrent table reservation and availability engine for 100,000+ restaurants, supporting 5M+ monthly reservations.

- **Functional:** Real-time search by location/time/party size, 5-minute holds, multi-table combinations, walk-in support, and 15-minute auto-release.
- **Non-Functional:** Strict Serializability (Zero double-bookings), Read Latency < 100ms, Write/Commit Latency < 200ms, handling extreme dinner service peak spikes.

2. Core API Contracts

To support the UI seamlessly, we expose three main endpoints:

Endpoint	Method	Description & Payload
/api/v1/availability/search	GET	Fetches available time slots. Query params: restaurant_id, party_size, date, time_window.
/api/v1/reservations/hold	POST	Places a temporary 5-min lock on table(s). Payload requires idempotency key.
/api/v1/reservations/confirm	POST	Commits the hold into a definitive reservation. Expects guest details & hold_token.

3. Database Schema & Data Modeling

A relational database like PostgreSQL is essential here to leverage ACID properties for financial-grade booking constraints. We use discrete time-slot partitioning instead of timestamp ranges to optimize indexing.

```
TABLE Restaurant {
  id UUID PRIMARY KEY,
  name VARCHAR,
  timezone VARCHAR,
  open_time TIME,
  close_time TIME
}

TABLE Physical_Table {
  id UUID PRIMARY KEY,
  restaurant_id UUID,
  min_capacity INT,
  max_capacity INT,
  is_combinable BOOLEAN
}

TABLE Virtual_Table {
  -- Represents combinable physical tables (e.g. Table 1 + Table 2)
  id UUID PRIMARY KEY,
  restaurant_id UUID,
  physical_table_ids ARRAY[UUID],
  combined_capacity INT
}

TABLE Table_Slot {
  -- Divides the day into discrete 15-minute chunks for O(1) lookups
  restaurant_id UUID,
  table_id UUID,   -- Can be physical or virtual
  slot_time TIMESTAMP, -- e.g., 2026-10-02 18:00:00
  status ENUM('AVAILABLE', 'HELD', 'BOOKED'),
  PRIMARY KEY (restaurant_id, table_id, slot_time)
}
```

```
TABLE Reservation {
 id UUID PRIMARY KEY,
 restaurant_id UUID,
 user_id UUID,
 table_ids ARRAY[UUID],
 start_time TIMESTAMP,
 end_time TIMESTAMP,
 status ENUM('HOLD', 'CONFIRMED', 'SEATED', 'CANCELLED', 'NO_SHOW')
}
```

4. Staff-Level Deep Dives & Architectural Trade-offs

4.1 Concurrency Control: The Double-Booking Problem

The Problem: At 5:00 PM, 50 users try to book the last available 4-seater at a popular restaurant for 7:00 PM.

The Solution (Distributed Redis Locking + DB Optimistic Concurrency):

1. **Phase 1: Fast Rejection (Redis):** When a user selects a time, the server attempts to acquire a Redis lock utilizing the key pattern lock:{restaurant_id}:{table_id}:{slot_time} with a TTL of 5 minutes (300s). If the lock is already taken, the request fails instantly (< 10ms latency).
2. **Phase 2: Database Hold Commit (PostgreSQL):** Once the Redis lock is acquired, the backend updates the Table_Slot status using an atomic row-level query: UPDATE Table_Slot SET status = 'HELD' WHERE table_id = X AND slot_time = Y AND status = 'AVAILABLE';. If this returns 0 updated rows, we release the Redis lock and fail.
3. **Phase 3: Final Confirmation:** User enters details and pays/confirms within 5 minutes. The Reservation is marked as CONFIRMED, and the Table_Slot is updated to BOOKED.

4.2 Time-Slot Partitioning vs. Range Queries

The Problem: Running standard overlapping range queries (e.g., WHERE new_start < existing_end AND new_end > existing_start) across millions of rows causes index scans and massive DB CPU spikes.

The Solution: Discrete 15-Minute Epoch Blocks.

We pre-generate Table_Slot entries for 3 months into the future. A 1-hour reservation simply locks four consecutive 15-minute rows. Finding availability becomes a lightning-fast precise match query: SELECT table_id FROM Table_Slot WHERE restaurant_id = X AND slot_time IN

(T1, T2, T3, T4) AND status = 'AVAILABLE' GROUP BY table_id HAVING count(*) = 4;. This query uses a B-Tree composite primary key index perfectly.

4.3 Dynamic Table Combination Algorithm

The Problem: A party of 8 searches for a table, but the restaurant only has 4-seaters left. We need to seamlessly combine Table 12 and Table 13.

The Solution: Read-Time Virtualization vs. Write-Time Shadowing

- Doing graph traversals (DFS/BFS) on a table-adjacency matrix at *read-time* violates our < 100ms search latency requirement.
- **Optimal Approach: Write-Time Shadowing.** Restaurant staff configure adjacent tables (e.g., T12 + T13). The system creates a Virtual_Table entity (e.g., VT_12_13) with capacity 8.
- When VT_12_13 is booked, an asynchronous Database Trigger or Application-level transaction automatically applies shadow locks to physical slots for T12 and T13.
- Conversely, if a party of 2 books T12, the system automatically shadows VT_12_13 as unavailable. Search queries remain O(1).

4.4 Walk-Ins & 15-Minute Grace Period (Auto-Release)

The Problem: If a confirmed party doesn't show up 15 minutes past their reservation time, the restaurant loses revenue. We must auto-release the table.

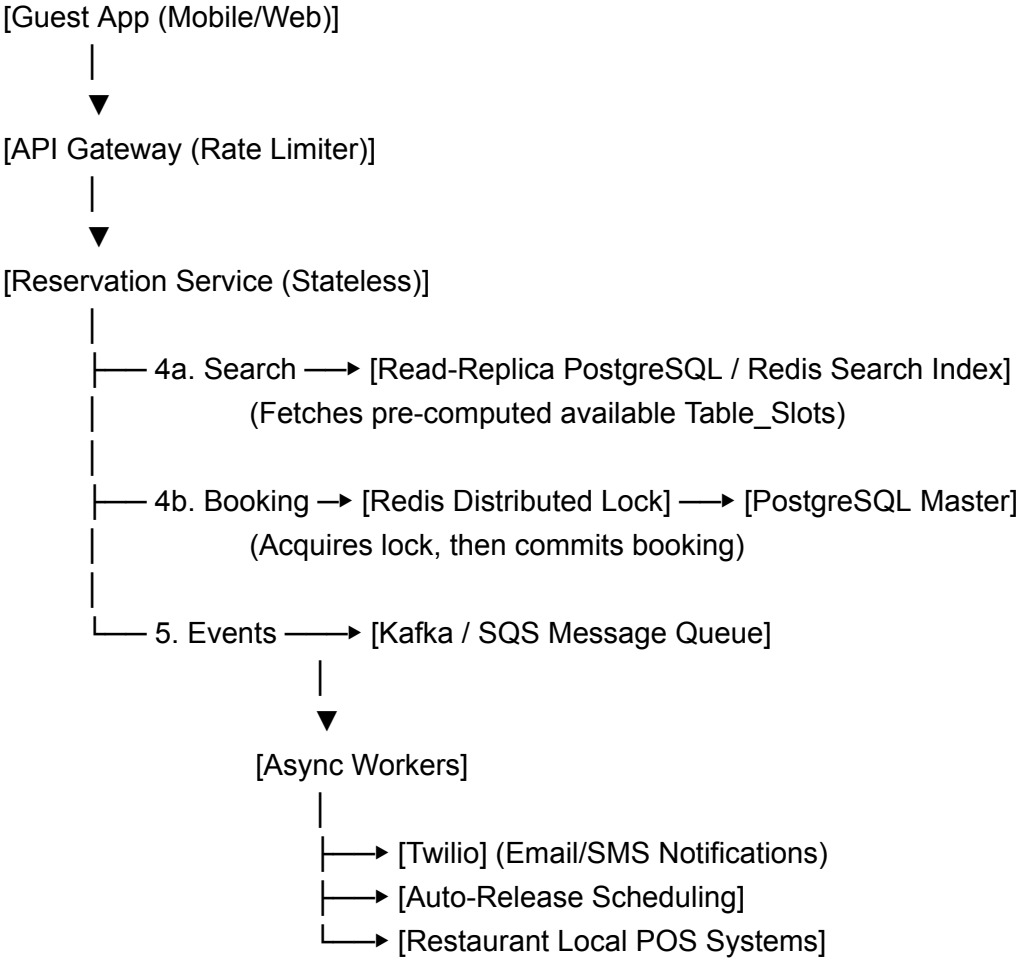
The Solution: Event-Driven Delay Queues

- We avoid running a global cron job that scans the database every minute (anti-pattern: database polling).
- Instead, when a reservation is confirmed, we publish an event to an **AWS SQS Delay Queue** (or RabbitMQ Dead Letter Exchange) with a visibility delay set exactly to Reservation_Start_Time + 15 minutes.
- When the message becomes visible, a worker consumes it. It checks the DB: SELECT status FROM Reservation WHERE id = X;
- If the status is still CONFIRMED (meaning the host hasn't marked them as SEATED via the POS walk-in app), the worker updates the status to NO_SHOW and executes an un-lock query on the Table_Slot rows, instantly returning the table to the available inventory pool.

5. System Architecture Flow Diagram (Mental Map)

1. Guest App (Mobile/Web) → 2. API Gateway (Rate Limiter) → 3. Reservation Service

(Stateless Microservice)
4a. Search: Reservation Service queries Read-Replica PostgreSQL or a Redis search index for pre-computed available Table_Slots.
4b. Booking: Reservation Service acquires Redis Distributed Lock, then commits to PostgreSQL Master.
5. Async Workers: Listen to Reservation Service events (via Kafka/SQS) to handle notifications (Email/SMS via Twilio), Auto-Release scheduling, and sync to Restaurant Local POS systems.



Here is the complete **Q&A and Deep-Dive Supplement**

Section 6: Follow-Up Deep-Dive Q&A

Question 1: Timezone Handling Across International Borders

Q: Why store `timezone` as `VARCHAR`? How do you handle a user in New York booking a restaurant in Paris? How is it rendered for the guest vs. restaurant staff?

A:

1. **Why `VARCHAR` for Timezone:** We store IANA time zone identifiers (e.g., `"Europe/Paris"`, `"America/New_York"`) instead of numeric UTC offsets (e.g., `+02:00`). Fixed offsets fail to account for Daylight Saving Time (DST) transitions. Standard IANA strings allow database and runtime libraries to resolve dynamic DST shifts automatically.
2.
3. **Cross-Timezone Booking Flow:**
4.
    - **Database Layer:** All timestamp fields (`Table_Slot.slot_time`, `Reservation.start_time`) are strictly stored in **UTC** (`TIMESTAMP WITH TIME ZONE` or UTC Unix Epoch milliseconds).
    -
    - **Backend Resolution:** When a user in New York ($UTC - 5$) requests "7:00 PM on Oct 10" for a Paris restaurant ($UTC + 2$), the API gateway takes the restaurant's IANA timezone (`Europe/Paris`), converts 7:00 PM local Paris time to its UTC equivalent (`17:00:00 UTC`), and executes the availability query in UTC.
    -
5. **UX (Guest) vs. RX (Restaurant Staff):**
6.
    - **Guest Mobile/Web App:** Displays times in local restaurant time with explicit context:
    -
    - *"7:00 PM CEST (Local Restaurant Time) — 1:00 PM EDT (Your Time)."*
    -
    - **Restaurant Staff (POS / KDS):** Always rendered in the restaurant's physical local time (`7:00 PM CEST`). Staff never interact with UTC or foreign customer timezones.
    -

Question 2: The `Table_Slot` Logic & Multi-Slot Bookings

Q: What is the thought process behind 15-minute `Table_Slot` discrete blocks? How are 2-hour reservations handled? How do I explain this in an interview?

A:

1. **Thought Process:** Traditional reservation engines evaluate availability using overlapping interval range queries (`WHERE new_start < existing_end AND new_end > existing_start`). Range scans fail to utilize standard B-Tree index lookups efficiently and trigger wide index-range locks in relational databases, resulting in deadlocks and latency spikes under heavy traffic. Discretizing time into 15-minute epoch buckets converts range scans into **exact-match set operations** ($O(1)$ index lookups).
2.
3. **Multi-Slot Booking Execution:**
4.
    - Turn times are dictated by party size (e.g., Party of 4 = 120 minutes = 8 consecutive 15-minute slots).
    -
    - Booking a 7:00 PM table executes a single atomic batch update across 8 deterministic rows:
    -

SQL

None

```
UPDATE Table_Slot
SET status = 'HELD', updated_at = NOW()
WHERE restaurant_id = 'rest_123'
  AND table_id = 'table_456'
  AND slot_time IN (
    '2026-10-02 19:00:00Z', '2026-10-02 19:15:00Z',
    '2026-10-02 19:30:00Z', '2026-10-02 19:45:00Z',
    '2026-10-02 20:00:00Z', '2026-10-02 20:15:00Z',
    '2026-10-02 20:30:00Z', '2026-10-02 20:45:00Z'
  )
  AND status = 'AVAILABLE';
```

```
```

- If the query returns `updated_rows = 8`, the lock succeeds. If it returns `< 8`, at least one slot was taken; the transaction immediately rolls back.
-

3. **Interview Pitch Script:**
4.

5. *"To avoid expensive database range scans that lock index pages under peak loads, I discretize time into 15-minute slots. A 2-hour reservation translates into an atomic set-update of 8 deterministic rows. This reduces availability checks from $O(N)$ interval comparisons down to an $O(1)$ primary key lookup."*
6.

Question 3: Justifying `Virtual_Table` over Read-Time Dynamic Traversal

Q: How do I justify choosing pre-configured Virtual Tables over dynamic graph traversals (DFS/BFS) during availability search?

A:

1. **Read Path Latency SLA (<100ms):** Restaurant search is 99% reads and 1% writes. Running DFS/BFS graph traversals over physical table adjacency matrices at read-time consumes high CPU overhead and causes search latency to scale linearly with table count.
2.
3. **Eliminating Multi-Resource Lock Deadlocks:** dynamically selecting and locking individual physical tables on the fly leads to multi-resource circular wait deadlocks when multiple users compete for adjacent tables simultaneously.
4.
5. **Write-Time Shadowing:** Pre-defined `Virtual_Table` entities maintain $O(1)$ search lookups. When a physical table is booked, a shadow write marks its parent `Virtual_Table` as unavailable. Conversely, booking a `Virtual_Table` shadow-locks its underlying physical table slots.
6.
7. **Interview Pitch Script:**
8.

9. *"Dynamic graph traversal during live search violates our 100ms latency target and*

introduces locking deadlocks under peak traffic. By utilizing pre-configured Virtual Tables with Write-Time Shadowing, we keep search queries as fast $O(1)$ lookups while enforcing strict consistency across physical and combined tables."

10.

Question 4: Data Modeling — Storing `table_ids` as Arrays vs. Junction Tables

Q: Is storing `table_ids ARRAY[UUID]` safe in PostgreSQL? What are the trade-offs vs. normalized junction tables?

A:

1. **Array Column (`UUID[]`) Limitations:**
2.
    - **Violates First Normal Form (1NF).**
    -
    - **No Native Foreign Key Constraints:** PostgreSQL cannot enforce referential integrity on elements inside an array. If a table row is deleted from `Physical_Table`, PostgreSQL cannot prevent dangling UUIDs in the array.
    -
    - **Indexing Overhead:** Requires GIN (Generalized Inverted Index) instead of standard B-Tree indexes for reverse lookups (e.g., finding all reservations for a physical table).
    -
3. **Production-Grade Schema (Normalized Junction Table):**
4.

SQL

None

```
-- Core Reservation Entity
CREATE TABLE Reservation (
    id UUID PRIMARY KEY,
    restaurant_id UUID NOT NULL REFERENCES
Restaurant(id),
    user_id UUID NOT NULL,
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,
```

```
    end_time TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(20) NOT NULL -- 'HOLD', 'CONFIRMED',
'SEATED', 'CANCELLED'
);

-- Normalized Junction Table (Many-to-Many Relationship)
CREATE TABLE Reservation_Table (
    reservation_id UUID NOT NULL REFERENCES
Reservation(id) ON DELETE CASCADE,
    table_id UUID NOT NULL REFERENCES Physical_Table(id),
    PRIMARY KEY (reservation_id, table_id)
);

CREATE INDEX idx_reservation_table_table_id ON
Reservation_Table(table_id);
```

Question 5: Java Implementation of Read-Time Dynamic Graph Traversal (DFS/BFS)

Q: How would dynamic table combination code look in Java, and why is this approach rejected for live search paths?

A:

Java

None

```
import java.util.*;

public class TableCombinationEngine {

    public static class TableNode {
        String id;
```

```
        int capacity;
        boolean isAvailable;
        List<TableNode> neighbors; // Physical adjacency
list

        public TableNode(String id, int capacity) {
            this.id = id;
            this.capacity = capacity;
            this.isAvailable = true;
            this.neighbors = new ArrayList<>();
        }
    }

    /**
     * Traverses physical table adjacencies to find a
valid combination.
     * Complexity: O(V + E) per search request (V =
tables, E = adjacency links)
     */
    public static List<TableNode>
findCombinedTablesBFS(TableNode startTable, int
partySize) {
        List<TableNode> selectedTables = new
ArrayList<>();
        Set<String> visited = new HashSet<>();
        Queue<TableNode> queue = new LinkedList<>();

        queue.add(startTable);
        visited.add(startTable.id);

        int currentCapacity = 0;
```

```
        while (!queue.isEmpty() && currentCapacity <
partySize) {
            TableNode current = queue.poll();

            if (current.isAvailable) {
                selectedTables.add(current);
                currentCapacity += current.capacity;

                if (currentCapacity >= partySize) {
                    return selectedTables; // Found
combination
                }

                for (TableNode neighbor :
current.neighbors) {
                    if (!visited.contains(neighbor.id) &&
neighbor.isAvailable) {
                        visited.add(neighbor.id);
                        queue.add(neighbor);
                    }
                }
            }
        }

        return currentCapacity >= partySize ?
selectedTables : Collections.emptyList();
    }
}
```

- **Why Rejected in Production:** $O(V + E)$ graph traversal across hundreds of tables for thousands of concurrent search requests wastes CPU cycles and introduces race

conditions between search phase and lock acquisition.

-

Question 6: High Traffic Spikes & Drop Days (Thundering Herd)

Q: How do you handle 100,000 users competing for 20 tables at 9:00 AM when a famous restaurant opens monthly booking drops?

A:

1. **API Gateway Rate Limiting:** Apply Token Bucket / Leaky Bucket rate limiting per IP/User ID at the Gateway to block bot scripts.
2.
3. **Redis Lua Script Counter (Fast In-Memory Guard):** Before hitting PostgreSQL, check an atomic counter in Redis (`DECR restaurant:{id}:available_slots`). If the counter reaches zero, drop incoming hold requests immediately at the edge (<5ms) with a "Fully Booked" response without querying the database.
4.
5. **Database Connection Pooling:** Utilize **PgBouncer** in transaction-pooling mode to prevent backend microservices from exhausting PostgreSQL database connection pools under traffic surges.
6.
7. **Asynchronous Reservation Queuing:** Route hold requests through a high-throughput Kafka topic or SQS queue. Worker pools process bookings sequentially, pushing confirmation or rejection status back to the client via WebSockets.
8.

Question 7: Distributed Locking Failure Modes & GC Pauses

Q: What happens if a worker thread experiences a long Garbage Collection (GC) pause while holding a Redis lock, causing the lock to expire before the database commit finishes?

A:

1. **The Risk:** Worker A acquires a 5-minute Redis hold lock. A 30-second JVM STW (Stop-The-World) GC pause occurs. The Redis lock TTL expires. Worker B acquires the lock for the same slot. Worker A wakes up and commits to the DB, causing a double-booking.
2.
3. **Mitigation 1: Optimistic Conditional Updates in DB (Final Source of Truth):** Redis is used solely as an intentional fast-rejection layer, not the authority for correctness. The PostgreSQL update relies on explicit atomic constraints:

4. `UPDATE Table_Slot SET status = 'HELD' WHERE table_id = X AND slot_time = Y AND status = 'AVAILABLE';`
5. Even if Worker A's Redis lock expired, PostgreSQL guarantees atomic row-level execution. If Worker B committed first, Worker A's update returns `0 updated rows` and fails safely.
6.
7. **Mitigation 2: Fencing Tokens:** Generate a monotonically increasing token (e.g., Redis `INCR fence_token`) upon acquiring the lock. The database validates that the incoming update carries the highest observed fencing token before committing.
8.

Question 8: Cache Invalidation & Change Data Capture (CDC)

Q: How do you keep search cache layers in sync without showing stale availability listings?

A:

1. **Avoid Application-Level Cache Updates:** Having application microservices update both DB and Redis directly leads to split-brain states when one network call succeeds and the other fails.
2.
3. **Change Data Capture (CDC) via Debezium & Kafka:**
4.
    - Primary commits occur strictly in PostgreSQL.
    -
    - **Debezium** tail-reads the PostgreSQL Write-Ahead Log (WAL) for state updates on `Table_Slot`.
    -
    - DB updates automatically publish CDC events to a Kafka topic.
    -
    - Lightweight worker consumers ingest CDC events and update or invalidate the corresponding key in Redis in near real-time (<20ms).